Міністерство освіти і науки України
Національний технічний університет України «КПІ імені Ігоря Сікорського»
Кафедра Інформаційних Систем та Технологій

Звіт
Лабораторної роботи №1
з дисципліни **«Розробка мобільних застосувань під Android»**
на тему **«дослідження роботи з елементами керування»**

Перевірив:
Старший викладач
Орленко Сергій Петрович

Виконав:
студент гр. ІК-33
Ткачук Олексій

Київ – 2026

Мета роботи: дослідити створення простого застосунку під платформу Андроїд та набути практичні навички з використання елементів керування інтерфейсу, мов програмування Java чи Kotlin.

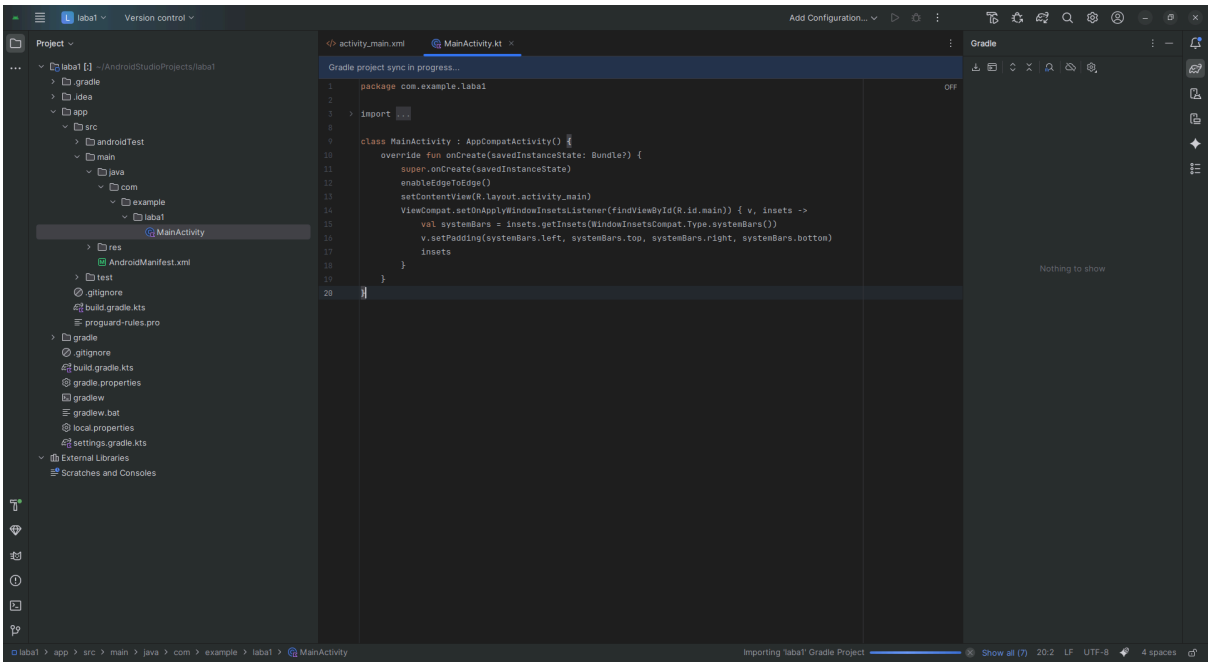
Завдання Варіант 7

Написати програму під платформу Андроїд, яка має інтерфейс для введення або/та вибору даних згідно варіанту (таблиця) і відображає результат взаємодії з цим інтерфейсом у деяке текстове поле цього інтерфейсу. Передбачити наступне: якщо не всі дані введені або обрані, а користувач натискає кнопку для отримання результату, то відобразити вікно, що спливає, з повідомленням завершити введення всіх даних.

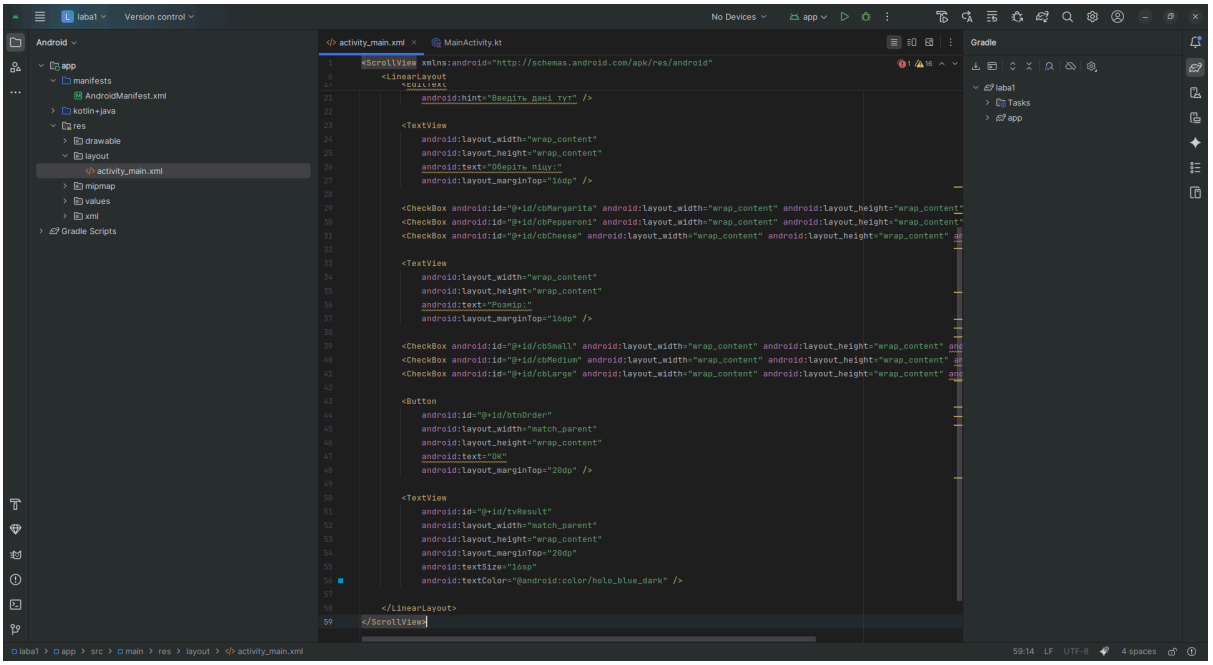
Вікно замовлення піци містить: текстове поле, групи прапорів (тип, розмір тощо), тобто чек-бокси, та кнопку «ОК». Вивести інформацію щодо замовлення при натисканні на кнопку «ОК» у деяке текстове поле.

Хід роботи

1. Створив новий проєкт з шаблоном "Empty Views Activity".



2. **Інтерфейс**:
    - Додав EditText для введення адреси або імені замовника.
    - Використав CheckBox для вибору типу піци (Маргарита, Пепероні, Чотири сири). Чекбокси дозволяють обрати кілька варіантів одразу (що логічно для піци).
    - Використав ще одну групу CheckBox для вибору розміру (25см, 30см, 40см).
    - Додав кнопку Button з текстом "OK".
    - Додав великий TextView внизу для виведення фінального результату замовлення.



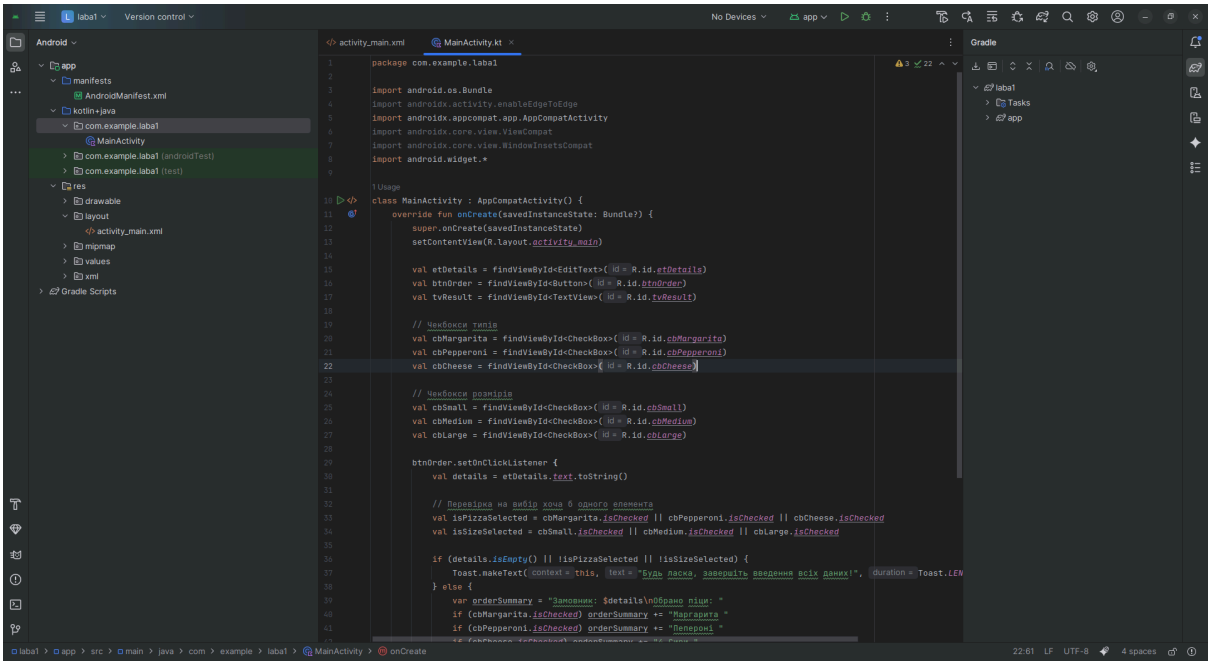

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:padding="16dp">



    <LinearLayout
```

```
        android:orientation="vertical"

        android:layout_width="match_parent"

        android:layout_height="wrap_content">



        <TextView

            android:layout_width="wrap_content"

            android:layout_height="wrap_content"

            android:text="Ваше ім'я/адреса:"

            android:textSize="18sp" />



        <EditText

            android:id="@+id/etDetails"

            android:layout_width="match_parent"

            android:layout_height="wrap_content"

            android:hint="Введіть дані тут" />



        <TextView

            android:layout_width="wrap_content"

            android:layout_height="wrap_content"

            android:text="Оберіть піцу:"

            android:layout_marginTop="16dp" />



        <CheckBox android:id="@+id/cbMargarita"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Маргарита" />
```

```
        <CheckBox android:id="@+id/cbPepperoni"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Пепероні" />

        <CheckBox android:id="@+id/cbCheese"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="4 Сири" />



        <TextView

            android:layout_width="wrap_content"

            android:layout_height="wrap_content"

            android:text="Розмір:"

            android:layout_marginTop="16dp" />



        <CheckBox android:id="@+id/cbSmall"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Мала (25см)" />

        <CheckBox android:id="@+id/cbMedium"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Середня (30см)" />

        <CheckBox android:id="@+id/cbLarge"
android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Велика (40см)" />



        <Button

            android:id="@+id/btnOrder"

            android:layout_width="match_parent"

            android:layout_height="wrap_content"

            android:text="OK"
```

```
            android:layout_marginTop="20dp" />



        <TextView

            android:id="@+id/tvResult"

            android:layout_width="match_parent"

            android:layout_height="wrap_content"

            android:layout_marginTop="20dp"

            android:textSize="16sp"

            android:textColor="@android:color/holo_blue_dark" />



    </LinearLayout>

</ScrollView>
```

3. **Логіка (MainActivity.kt)**:
    - Прив'язав UI-елементи через findViewById.
    - Реалізував перевірку на порожні поля: якщо текст не введений або жоден чекбокс не обраний — програма виводить Toast (спливаюче вікно) з проханням завершити введення.
    - Створив алгоритм збору даних: програма перевіряє стан кожного чекбокса, формує рядок замовлення і виводить його в результуюче поле.

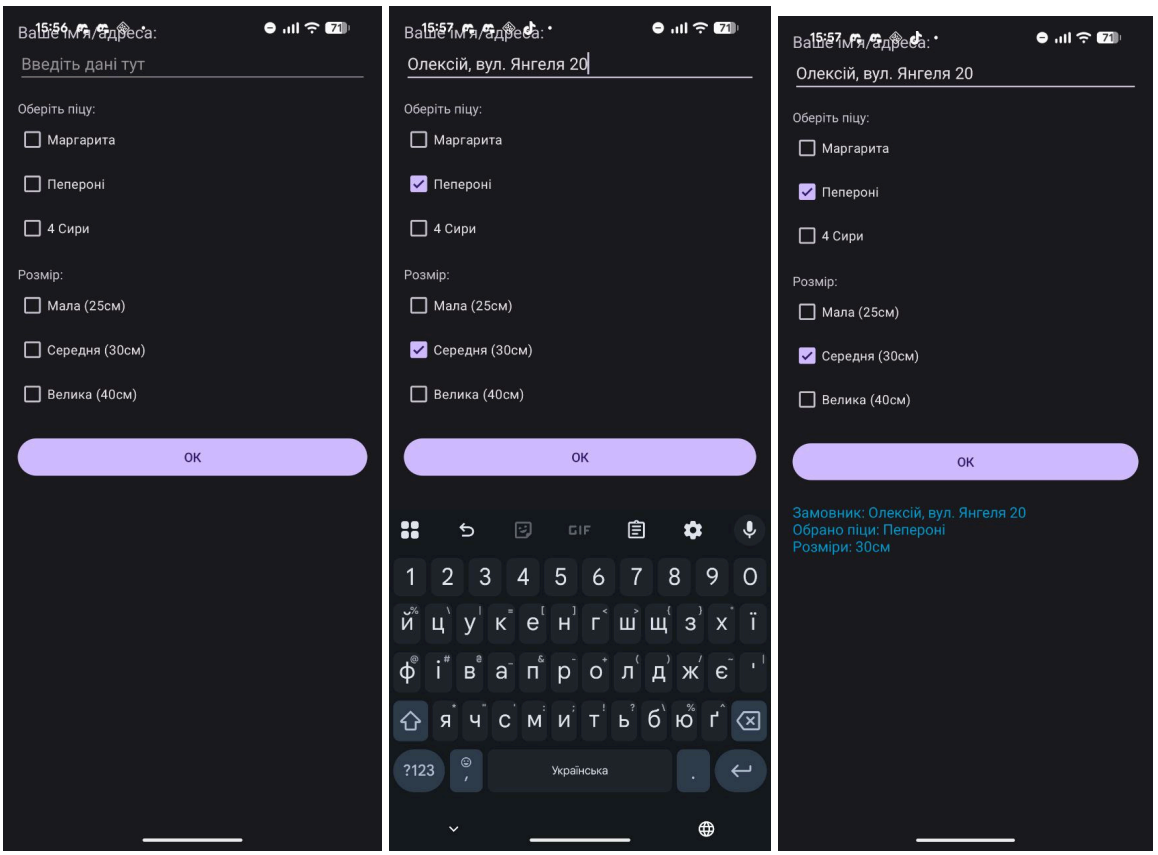



```
package com.example.laba1



import android.os.Bundle

import androidx.activity.enableEdgeToEdge

import androidx.appcompat.app.AppCompatActivity

import androidx.core.view.ViewCompat

import androidx.core.view.WindowInsetsCompat

import android.widget.*



class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {

       super.onCreate(savedInstanceState)

       setContentView(R.layout.activity_main)
```

```
        val etDetails = findViewById<EditText>(R.id.etDetails)

        val btnOrder = findViewById<Button>(R.id.btnOrder)

        val tvResult = findViewById<TextView>(R.id.tvResult)



        // Чекбокси типів

        val cbMargarita = findViewById<CheckBox>(R.id.cbMargarita)

        val cbPepperoni = findViewById<CheckBox>(R.id.cbPepperoni)

        val cbCheese = findViewById<CheckBox>(R.id.cbCheese)



        // Чекбокси розмірів

        val cbSmall = findViewById<CheckBox>(R.id.cbSmall)

        val cbMedium = findViewById<CheckBox>(R.id.cbMedium)

        val cbLarge = findViewById<CheckBox>(R.id.cbLarge)



        btnOrder.setOnClickListener {

            val details = etDetails.text.toString()



            // Перевірка на вибір хоча б одного елемента

                        val  isPizzaSelected  =  cbMargarita.isChecked  ||
cbPepperoni.isChecked || cbCheese.isChecked

             val isSizeSelected = cbSmall.isChecked || cbMedium.isChecked ||
cbLarge.isChecked



            if (details.isEmpty() || !isPizzaSelected || !isSizeSelected) {
```

```
                    Toast.makeText(this, "Будь ласка, завершіть введення всіх
даних!", Toast.LENGTH_SHORT).show()

            } else {

                var orderSummary = "Замовник: $details\nОбрано піци: "

                if (cbMargarita.isChecked) orderSummary += "Маргарита "

                if (cbPepperoni.isChecked) orderSummary += "Пепероні "

                if (cbCheese.isChecked) orderSummary += "4 Сири "



                orderSummary += "\nРозміри: "

                if (cbSmall.isChecked) orderSummary += "25см "

                if (cbMedium.isChecked) orderSummary += "30см "

                if (cbLarge.isChecked) orderSummary += "40см "



                tvResult.text = orderSummary

            }

        }

    }

}
```

4. Результат роботи



Висновок

Під час виконання лабораторної роботи опрацьовано базові інструменти Android-розробки, створення графічного інтерфейсу в XML, написання логіки на мові Kotlin. На прикладі програми для замовлення піци працював з текстовими полями та чекбоксами, а також реалізував перевірку введених даних, щоб програма не видавала помилок при порожніх полях.

Контрольні запитання

1. Архітектура застосунку під платформу Андроїд.

Архітектура Android побудована за принципом багатошарового стеку програмного забезпечення. В її основі лежить ядро Linux, що відповідає за низькорівневе керування пам’яттю, процесами та драйверами

апаратного забезпечення. Над ядром розташовується рівень апаратної абстракції (HAL), який надає стандартні інтерфейси для взаємодії з фізичними модулями пристрою. Наступним шаром є Android Runtime (ART) разом із нативними бібліотеками C/C++, що забезпечує виконання коду у форматі DEX. Вище знаходиться Java API Framework — каркас, який надає розробникам доступ до системних служб (Activity Manager, Resource Manager тощо). Найвищим рівнем є рівень системних та користувацьких додатків, які взаємодіють із нижніми шарами через згаданий API-каркас.

2. Загальний огляд компонентів застосунку під Андроїд.

Будь-який Android-застосунок базується на чотирьох основних типах компонентів, кожен з яких виконує свою роль. «Діяльності» (Activities) забезпечують візуальний інтерфейс та взаємодію з користувачем. «Служби» (Services) призначені для виконання тривалих операцій у фоновому режимі без прямої візуалізації. «Приймачі оголошень» (Broadcast Receivers) дозволяють додатку реагувати на загальносистемні події або сповіщення від інших програм. Нарешті, «Постачальники вмісту» (Content Providers) керують спільним набором даних додатка, забезпечуючи безпечний доступ до них іншим компонентам або стороннім застосункам.

3. Життєвий цикл компоненту «Діяльність» (Activity).

Життєвий цикл Activity являє собою набір станів, через які проходить вікно програми від моменту створення до знищення. Цей процес

регулюється системними функціями зворотного виклику. Метод onCreate() ініціалізує вікно, onStart() робить його видимим, а onResume() переводить у стан активної взаємодії. Якщо Activity частково прикривається іншим елементом, викликається onPause(), а при повному зникненні з екрана — onStop(). Кінцевим етапом є onDestroy(), що звільняє всі ресурси. Якщо користувач повертається до зупиненої діяльності, вона проходить через onRestart() та повторну активацію.

4. Життєвий цикл компоненту «Служба» (Service).

Життєвий цикл служби відрізняється залежно від способу її використання. Запущена служба (Started Service) ініціюється методом onStartCommand() і працює незалежно від компонента, що її викликав, до повної зупинки. Прив’язана служба (Bound Service) пропонує інтерфейс клієнт-серверної взаємодії: вона активується через onBind() і існує лише доти, доки до неї підключений хоча б один клієнт. Обидва типи служб починають роботу з методу onCreate() та завершують її через onDestroy(), де виконується фінальне очищення ресурсів.

5. Опис процесів платформи Андроїд.

Система Android керує ресурсами, надаючи кожному додатку окремий процес. Важливість процесу визначається його пріоритетністю, що базується на видимості для користувача. Процеси переднього плану (Foreground), де виконується активна діяльність, мають найвищий пріоритет. Далі йдуть видимі процеси (Visible), які не перебувають у фокусі, але відображаються на екрані. Нижчу ланку займають службові

процеси (Service) та фонові (Cached) процеси. У випадку дефіциту оперативної пам'яті система автоматично завершує процеси з найнижчим пріоритетом для стабільної роботи активних додатків.

6. Яким чином активуються компоненти застосунку.

Активація більшості компонентів (Activity, Service, Broadcast Receiver) здійснюється за допомогою асинхронних повідомлень, що називаються Intents (наміри). Явні наміри чітко вказують клас компонента, який слід запустити, тоді як неявні описують дію (наприклад, відкрити веб-сторінку чи надіслати лист), дозволяючи системі самостійно підібрати відповідний додаток. На відміну від них, Постачальники вмісту активуються через запити від ContentResolver, що ініціюються під час звернення до даних.

7. Призначення файлу маніфесту та його структура.

Файл AndroidManifest.xml виконує роль головного конфігураційного документа додатка, який зчитується системою під час встановлення. У ньому обов’язково декларуються всі компоненти застосунку, визначаються необхідні системні дозволи (наприклад, доступ до мережі) та вказуються метадані, такі як назва пакета та версія SDK. Структурно файл складається з кореневого тегу <manifest>, всередині якого розміщується блок <application>, що містить описи конкретних компонентів (теги <activity>, <service> тощо) та налаштування фільтрів намірів.

8. Поняття ресурсу та яким чином визначаються ресурси.

Ресурси в Android — це зовнішні елементи додатку, такі як текстові рядки, зображення, кольорові схеми та макети інтерфейсу, що відокремлені від програмного коду для забезпечення гнучкості розробки. Вони зберігаються у структурованих підпапках директорії res/. Кожен ресурс автоматично отримує унікальний числовий ідентифікатор у згенерованому класі R. Це дозволяє звертатися до них із програмного коду (через R.layout.main) або з інших XML-файлів (через @string/label), що значно спрощує локалізацію та адаптацію інтерфейсу під різні пристрої.